MH1301 Discrete Mathematics

Handout 11: Graph Theory (5): Minimum Spanning Trees, Shortest Paths

Synopsis

This is the last handout on graph theory. In this handout we cover

- Minimum Spanning Trees
- Shortest Path Problems.

Recall: Minimum Spanning Trees

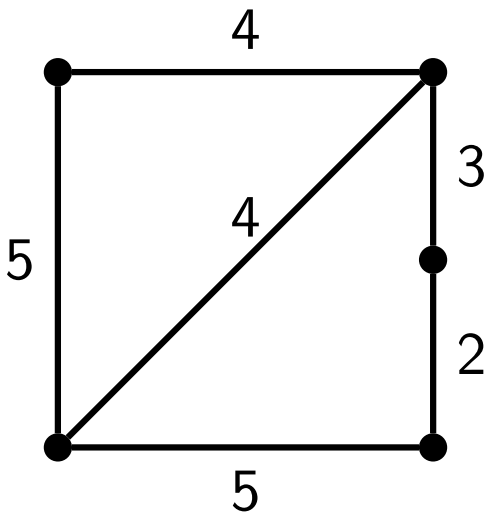
Spanning Trees (Definition 1 Section 11.4)

Let G be a connected simple graph. A spanning tree of G is a subgraph of G that is a tree containing all vertices of G.

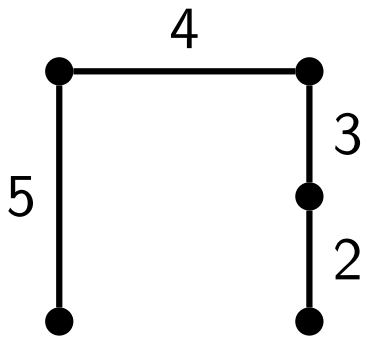
Minimum Spanning Trees (MST) (Definition 1 Section 11.5)

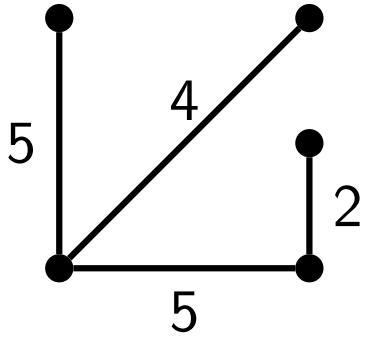
Given an edge-weighted graph G, a minimum spanning tree of G is a spanning tree that has the smallest total sum of weights.

Example

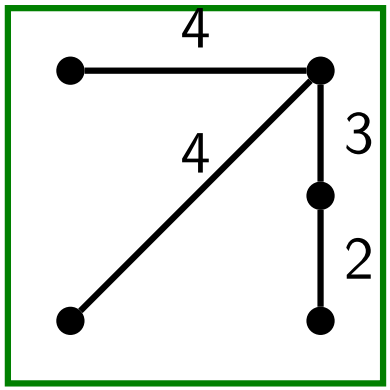


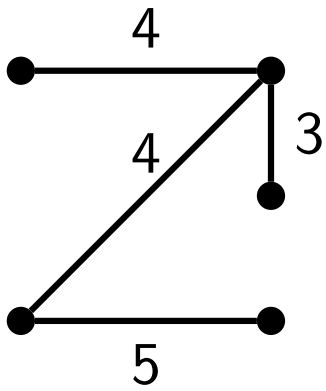
Graph G





MST





Question

How to find a minimum spanning tree in an edge-weighted graph?

Algorithms

- An algorithm is a procedure for performing a computation.
- Start from an initial state and an input (perhaps empty), eventually produce an output.
- An algorithm consists of **primitive steps/instructions** that can be executed mechanically.
- Important criteria to evaluate an algorithm:
  - **Correctness**: a must.
  - **Running time**: number of primitive steps needed to perform the computation.

Algorithms for MST: a Greedy Approach

comprehensive approach

Algorithm: SomeMSTAlgorithm(G):

Initialize $T = \emptyset$; // T will store edges of a MST

while T *is not a spanning tree of* G **do**

choose $e \in E$ that satisfies some condition;

add e to T;

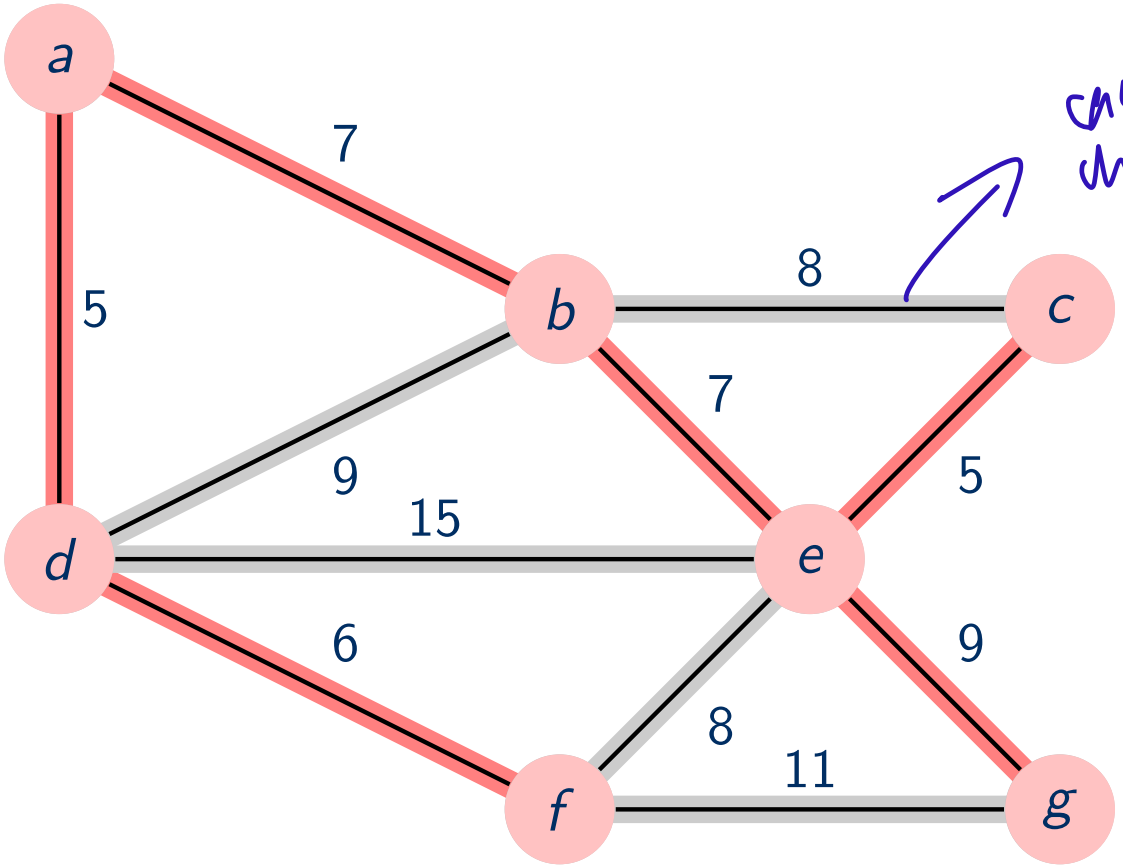
return T

Which edges, and in what order, should be processed and added to the spanning tree?

Kruskal's Algorithm

Kruskal's Algorithm

Process edges in the increasing order of their costs (break ties arbitrarily), and add edges to T as long as they don't form a cycle.



cannot choose grey edges, if not it forms a cycle

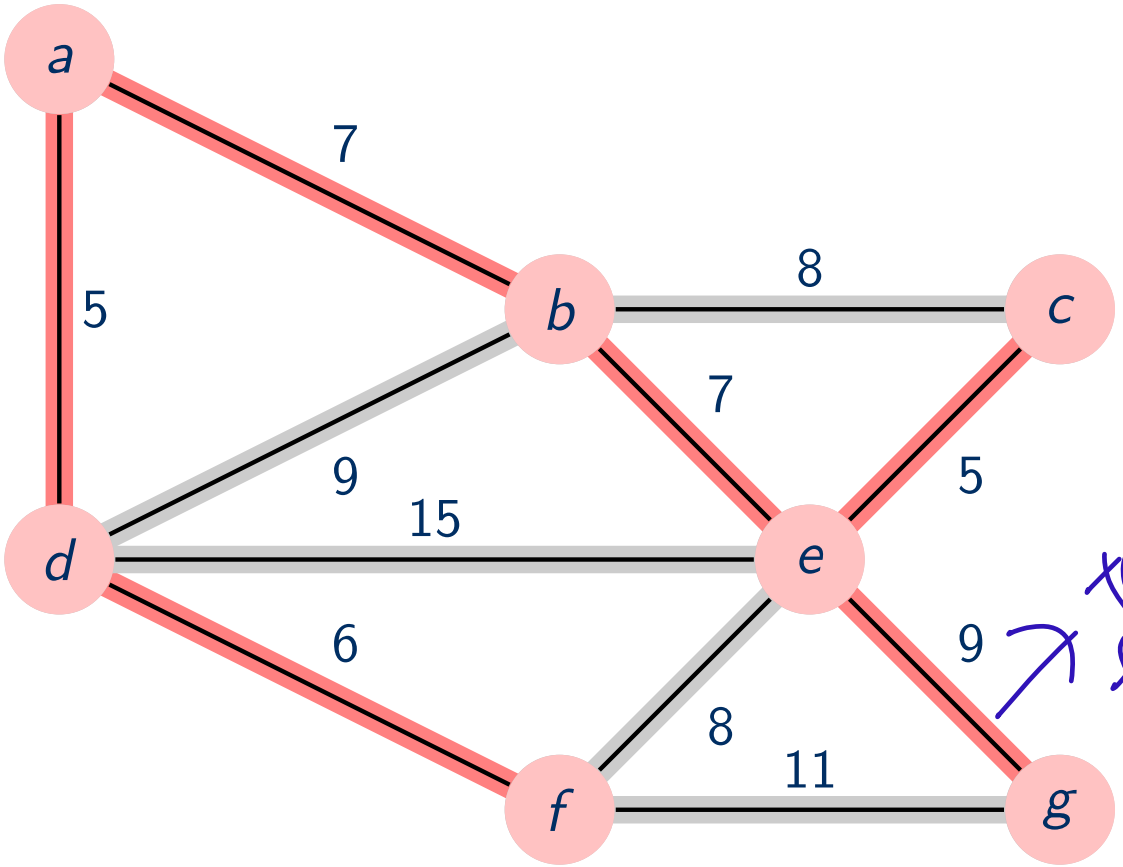
Edges added to the MST:

- $\{a, d\}$
- $\{c, e\}$
- $\{d, f\}$
- $\{a, b\}$
- $\{b, e\}$
- $\{e, g\}$

Prim's Algorithm

Prim's Algorithm

T maintained by the algorithm will be a tree, starting from a single vertex.
In each iteration, pick edges with least attachment cost to T.



Edges added to the MST:

- $\{a, d\}$
- $\{d, f\}$
- $\{a, b\}$
- $\{b, e\}$
- $\{c, e\}$
- $\{e, g\}$

Correctness of MST algorithms

Both Krusal's and Prim's algorithms are correct MST algorithms.

- Many other different MST algorithms.
- All of them rely on some basic properties of MSTs.

Assumption

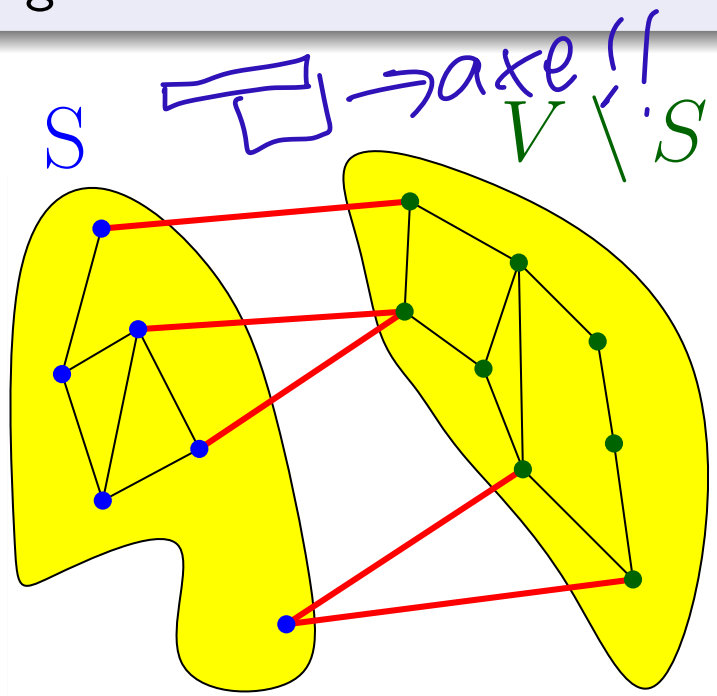
For simplicity, we assume that edge costs are distinct, that is no two edge costs are equal.

Cut

Definition

Given a graph $G = (V, E)$, a cut is a partition of the vertices into two disjoint subsets.

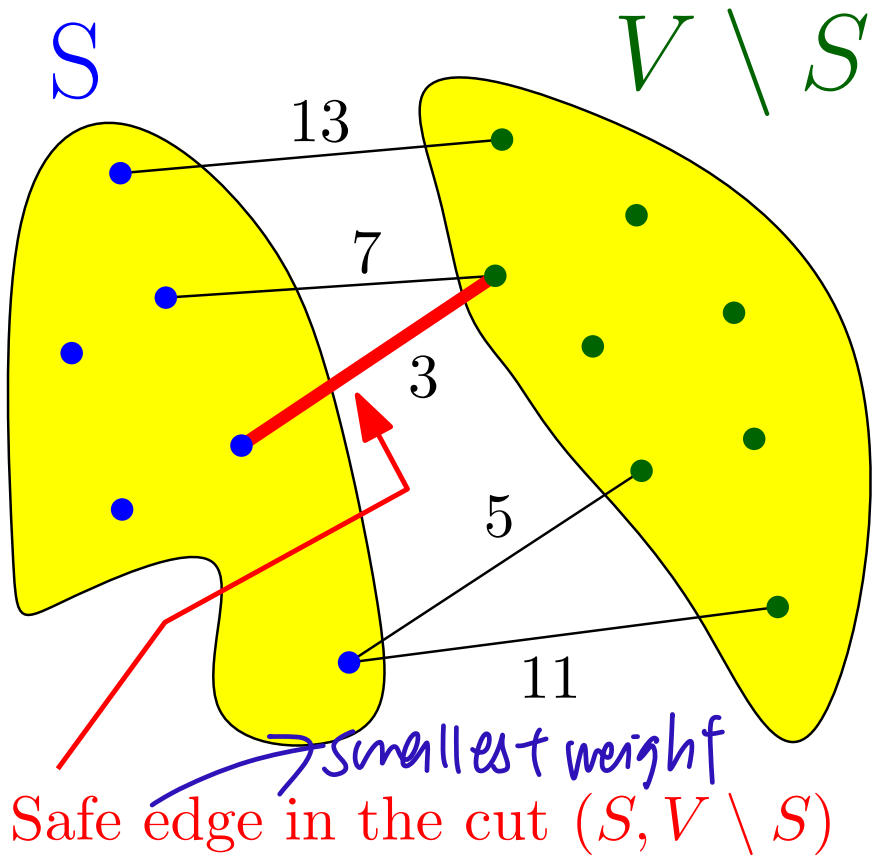
Edges that have one endpoint in each subset of the partition are the edges of the cut, and these edges are said to cross the cut.



Safe Edges

Safe Edges

An edge e is a safe edge if there exists a partition of V into S and $V \backslash S$, and e is the unique minimum cost edge crossing this partition.



Cut Property

Theorem

Given an undirected connected graph G with distinct edge costs, the set of safe edges in G form the unique MST of G.

Proof: Theorem is proved in two steps (stated as two lemmas):

1. Lemma1: If e is a safe edge, then every MST contains e.

$$\{\text{safe edges}\} \subseteq \text{MST}$$

2. Lemma 2: A MST does not contain other edges. Or equivalently, the set of safe edges form a connected graph that covers every vertex.

$$\{\text{safe edges}\} \supseteq \text{MST}$$

combined gives $\{\text{safe edges}\} = \text{MST}$

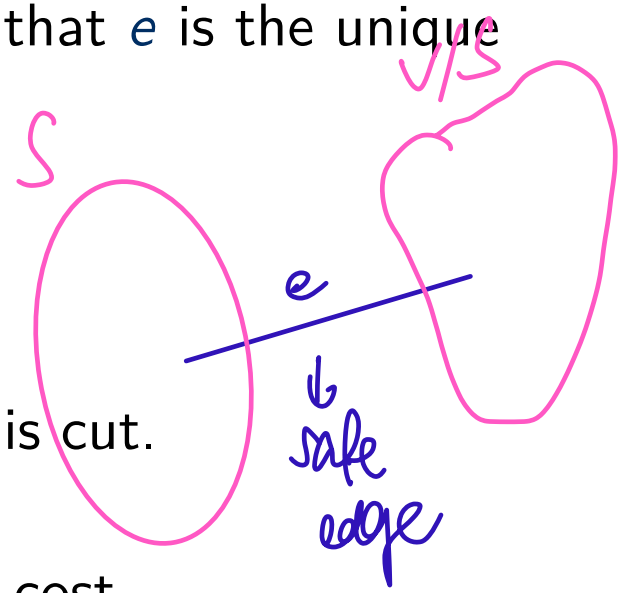
Cut Property

Lemma 1

If e is a safe edge, then every MST contains e.

Proof: Suppose not. I.e., suppose e is a safe edge that is not in some MST T.

- $e = \{u, v\}$ is safe $\implies$ there is an $S \subset V$ such that e is the unique minimum cost edge crossing S and $V - S$.
- Adding e to T creates a cycle C in $T \cup \{e\}$.
- There must exist another edge e' in T cross this cut.
- $T' = (T \backslash \{e'\}) \cup \{e\}$ is a spanning tree of lower cost.



□

Cut Property

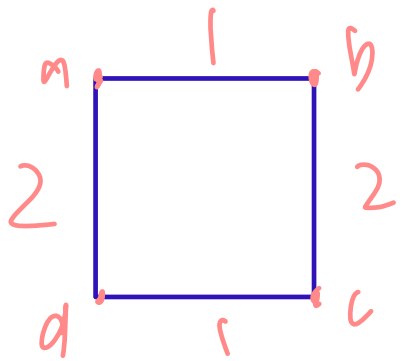
Lemma 2

The set of safe edges forms a connected graph that covers every vertex.

Proof:

- Suppose not. (I.e., there is some v not covered by the set of safe edges.) Let S be a connected component in the graph induced by safe edges.

- Let e be the smallest cost edge crossing S $\Longrightarrow$ e is a safe edge. □



Kruskal's Algorithm

Process edges in the increasing order of their costs (break ties arbitrarily), and add edges to T as long as they don't form a cycle.

Algorithm: Kruskal(G):

Initialize $T = \emptyset$; // T will store edges of a MST
while T *is not a spanning tree of* G **do**
    choose $e \in E$ of minimum cost;
    remove e from E;
    if $T \cup \{e\}$ *does not contain cycles* **then**
        add e to T;
return T

Correctness

Process edges in the increasing order of their costs (break ties arbitrarily), and add edges to T as long as they don't form a cycle.

Only need to show that all edges added are safe.

Proof:

- When $e = \{u, v\}$ is added to the tree, let S and S' be the connected components containing u and v respectively.
- e has minimum cost under all edges forming no cycle, hence e has minimum cost among all edges crossing the cut S (and also S')
- e is a safe edge.

□

Prim's Algorithm

T maintained by the algorithm will be a tree, starting from a single vertex. In each iteration, pick edges with least attached cost to T.

Algorithm: Prim(G):

Initialize $T = \emptyset$; // T will store edges of a MST

Initialize $S = \{1\}$;

while T *is not a spanning tree of* G **do**

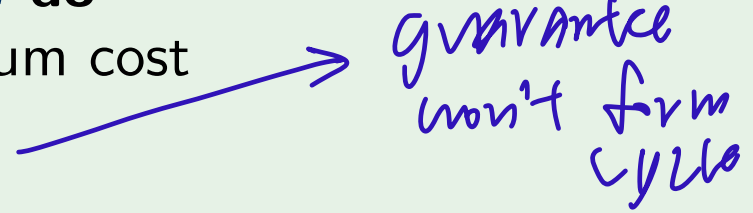
choose $e = (u, v) \in E$ of minimum cost

such that $u \in S$ and $v \in V - S$;

$T = T \cup \{e\}$;

$S = S \cup \{v\}$;

return T



Correctness

T maintained by the algorithm will be a tree, starting from a single vertex. In each iteration, pick edges with least attached cost to T.

Proof:

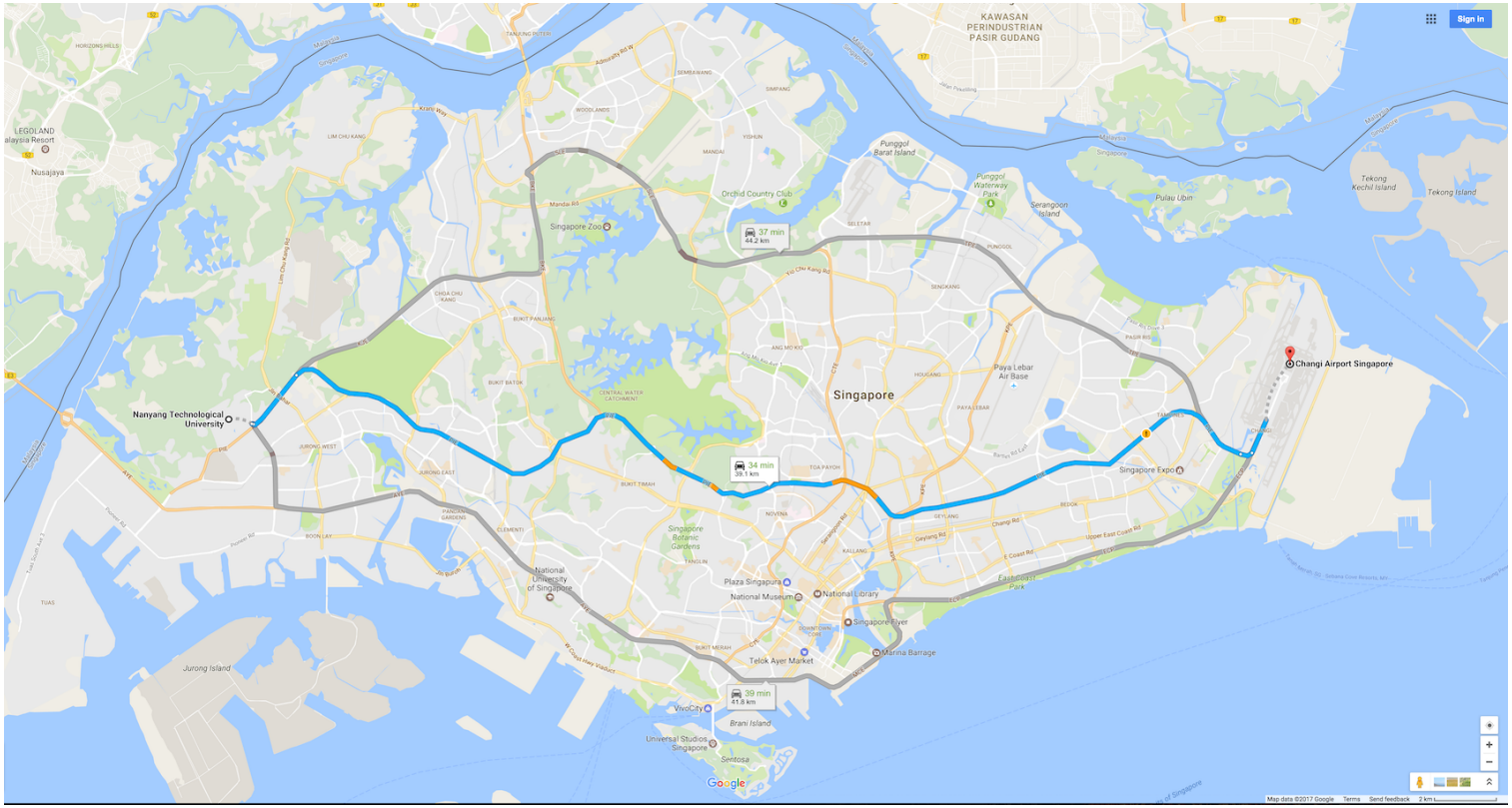
1. If e is added to the tree, then e is safe
   - Let S be the vertices connected by edges in T when e is added.
   - e is the minimum cost edge crossing cut $(S, V \backslash S)$.
2. S is connected in each iteration and eventually $S = V$.

□

Shortest Paths

Shortest Path

How to find the shortest route from NTU to Changi airport?



Shortest Paths

- In many problems one wants to find a shortest path in a weighted graph

  find lowest weight in a weighted graph

- Usually one wants to find a shortest path between two vertices u, v
- In this lecture all edge-weights are going to be non-negative
- Shortest paths are useful e.g. if
  - Edge weights are geographical distance
  - Edge weights are costs of using edges
  - Vertices are currencies and edge weights model exchange rates
  - etc.

Distances

Shortest Paths

For an edge-weighted graph $G = (V, E, W)$

- a shortest path between vertices u, v is a simple path that starts at u and ends at v such that the sum of edge weights on the path is minimal;
- this sum of edge weights is the length of the path;
- the distance between u and v is the length of a shortest path between u and v in G.
  - if there is no path that connects u and v, then the distance between them is ∞.

Shortest Paths Problems

Shortest Paths Problems

Given a (undirected or directed) weighted graph $G = (V, E)$ with edge lengths (or weights). For edge $e = (u, v)$, $\ell(e) = \ell(u, v)$ is its length.

1. Given vertices s, t, find a shortest path from s to t.
2. Given vertex s, find shortest paths from s to all other vertices.
3. Find shortest paths for all pair of vertices.

In this lecture we focus on problem (1) and (2).

Single-Source Shortest Path

Single-Source Shortest Path

Given a directed weighted graph $G = (V, E)$ with non-negative edge lengths (or weights). For edge $e = (u, v)$, $\ell(e) = \ell(u, v)$ is its length.

1. Given vertices s, t, find a shortest path from s to t.
2. Given vertex s, find shortest paths from s to all other vertices.

Focus on directed graphs.

- undirected graph problem can be reduced to directed graph problem

Single-Source Shortest Path

I think we ignore this slide.

Single-Source Shortest Path

Given a directed weighted graph $G = (V, E)$ with non-negative edge lengths (or weights). For edge $e = (u, v)$, $\ell(e) = \ell(u, v)$ is its length.

1. Given vertices s, t, find a shortest path from s to t.
2. Given vertex s, find shortest paths from s to all other vertices.

Algorithm Ideas

We explore vertices in increasing distance from s

Let $d(v)$ denote the shortest path length from s to v.

Algorithm: ShortestPathAlgorithm(s):

Initialize $d(v) = \infty$ for each vertex $v \in V$;
Initialize $S = \emptyset$;
while $S \neq V$ **do**
    Find vertex $v \in V - S$ that is the closest to s;
    Update $d(v)$;
    $S = S \cup \{v\}$

How to find the next closest vertex?

Finding the ith Closest Vertex

At the beginning of the ith iteration, S already stores the $i-1$ closest vertices to s.

What do we know about the ith closest vertex?

Claim

Let P be a shortest path from s to v where v is the ith closest vertex. Then all intermediate vertices in P belong to S.

Proof: Let v' be an intermediate vertex in path P, then $d(v') < d(v)$. Thus $v' \in S$.

reach v' before

Corollary

At each step, the next closest vertex is adjacent to S.

Finding the ith Closest Vertex

Lemma

If $s = v_0 \to v_1 \to \cdots \to v_k = v$ is a shortest path from s to v, then for any $1 \le i < k$:

- $s = v_0 \to v_1 \to \cdots \to v_i$ is a shortest path from s to v_i

Assume that S contains the $i-1$ closest vertices to s.

For each $u \in V - S$, let

$$\pi(u) = \min_{a \in S}(\mathrm{d(a)} + \ell(\mathrm{a}, \mathrm{u}))$$

length of edge $a-u$.

Lemma

If u is the ith closest vertex to s, then $\pi(u) = \mathrm{d(u)}$.

Corollary

The ith closest vertex to s is the vertex $u \in V - S$ such that $\pi(u) = \min_{v \in V-S} \pi(v)$.

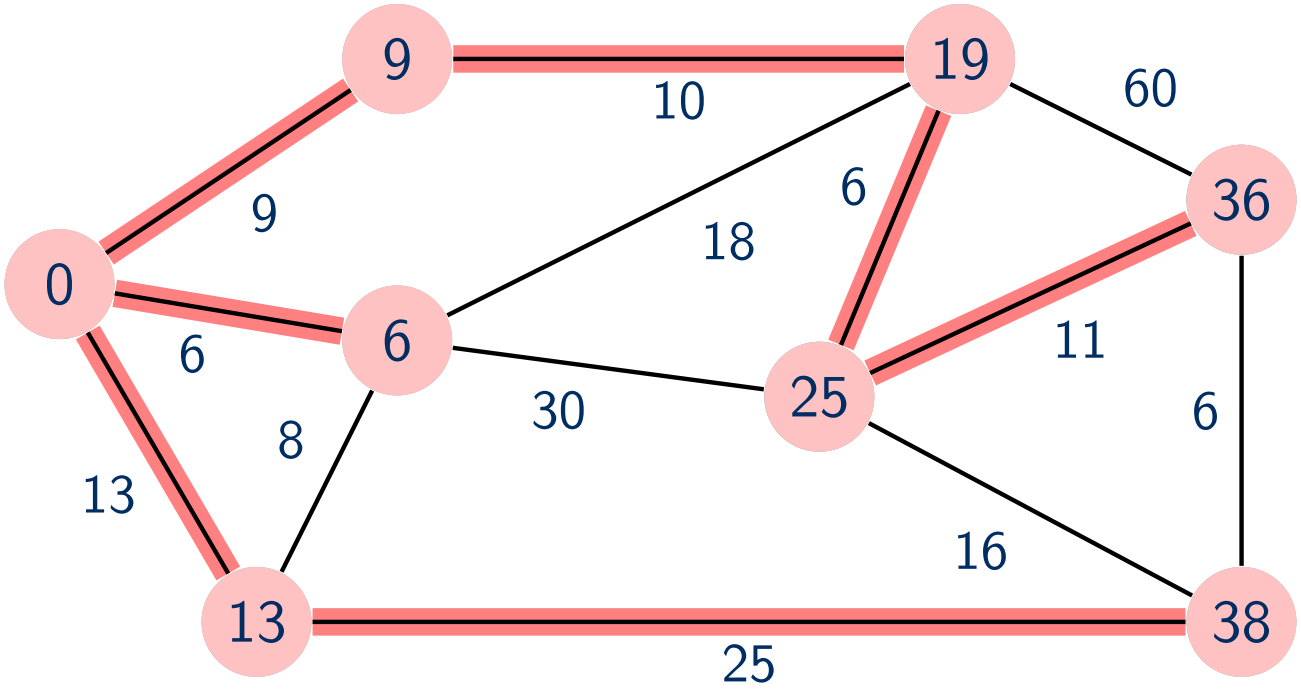
Algorithm

Algorithm: ShortestPathAlgorithm(s):

Initialize $\pi(v) = \infty$ for each vertex $v \in V$;
Initialize $S = \emptyset, \pi(s) = 0$;
while $S \neq V$ **do**
  Find vertex $u \in V - S$ such that $\pi(u) = \min_{v \in V-S} \pi(v)$;
  $d(u) = \pi(u)$;
  $S = S \cup \{u\}$;
  foreach *vertex* $v \in V - S$ **do**
    $\pi(v) = \min_{a \in S}(d(a) + \ell(a, v))$;

Correctness: by induction using previous lemmas.

Example



More Efficient Implementation

Critical Optimization For each unexplored vertex v, explicitly maintain $\pi(v)$ instead of computing directly from formula:

$$\pi(v) = \min_{u \in S}(\mathrm{d}(\mathrm{u}) + \ell(\mathrm{u}, \mathrm{v})).$$

- For each $v \notin S$, $\pi(v)$ can only decrease (because S can only increase).
- More specifically, suppose u is added to S and there is an edge (u, v) leaving u. Then, it suffices to update

$$\pi(v) = \min\{\pi(v), \mathrm{d}(\mathrm{u}) + \ell(\mathrm{u}, \mathrm{v})\}$$

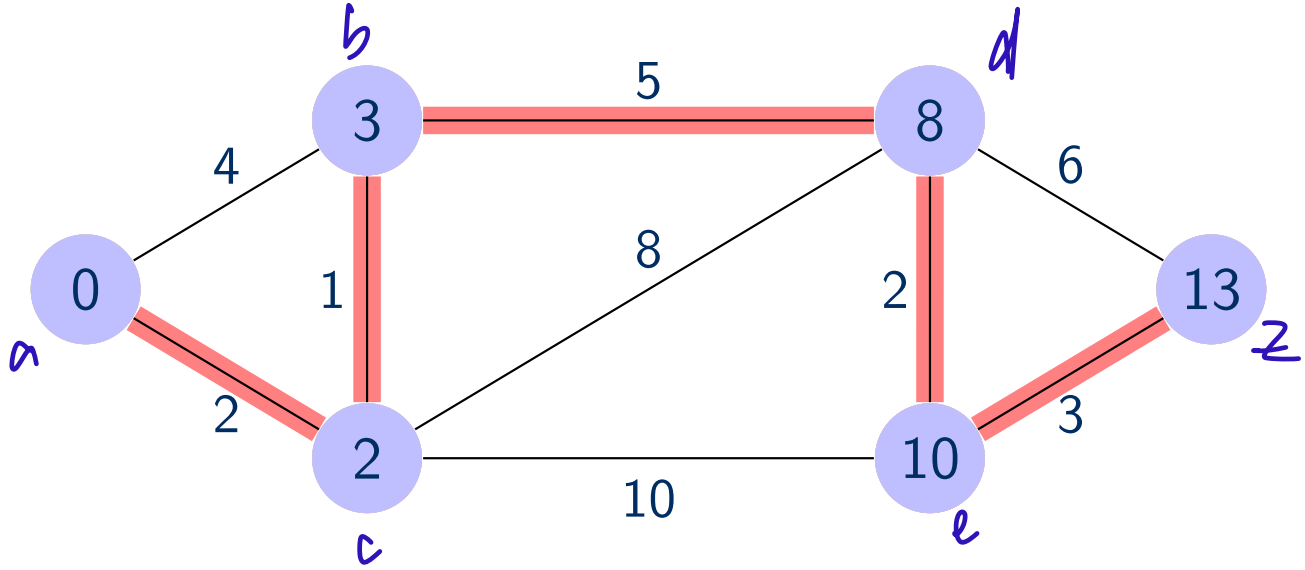
Dijkstra's Algorithm

1. eliminate $\pi(v)$ and let $d(v)$ maintain it
2. update d values after adding v by scanning edges out of v

Algorithm: Dijkstra(s):

Initialize $d(v) = \infty$ for each vertex $v \in V$;
Initialize $S = \emptyset, d(s) = 0$;
while $S \neq V$ **do**
    Find vertex $u \in V - S$ such that $d(u) = \min_{v \in V-S} d(v)$;
    $S = S \cup \{u\}$;
    foreach *vertex* $v \in Adj(u)$ **do**
        $d(v) = \min\{d(v), d(u) + \ell(u, v)\}$;

Example



	$d(a)$	$d(b)$	$d(c)$	$d(d)$	$d(e)$	$d(z)$	S
Round 0	0	∞	∞	∞	∞	∞	$\emptyset$
Round 1	0	4	2	∞	∞	∞	$\{a\}$
Round 2	0	3	2	10	12	∞	$\{a, c\}$
Round 3	0	3	2	8	12	∞	$\{a, c, b\}$
Round 4	0	3	2	8	10	14	$\{a, c, b, d\}$
Round 5	0	3	2	8	10	13	$\{a, c, b, d, e\}$
Round 6	0	3	2	8	10	13	$\{a, c, b, d, e, z\}$

Reading Assignment

- Rosen (7th edition):
  - Chapter 11.5. Minimum Spanning Trees
  - Chapter 10.6. Shortest-Path Problems